AI WORKLOAD PROFILER ON A RISC-V CORE

Fatimata Abdou Dramane

PROJECT OVERVIEW

AI models profiling and benchmarking help assess hardware efficiency and prevent production faults. The goal of this project is to take a foundational AI operator, like GEMM (General Matrix Multiply) or a LayerNorm / ReLU activation function, and profile its execution efficiency on a RISC-V core. Specifically, we will compare how standard scalar execution performs against RISC-V Vector Extensions (RVV), analyzing the exact hardware bottlenecks (cache misses, pipeline stalls, instruction count).

PROJECT BREAKDOWN

- Set Up the RISC-V Toolchain & Simulator
- Write the baseline (scalar) AI Operator
- Write the Optimized (Vector) AI Operator
- Profile the microarchitecture

SET UP THE RISC-V TOOLCHAIN & SIMULATOR

- Install the open-source **RISC-V GNU Toolchain** with vector support (rv64gcv).
- For simulation and deep profiling, use **Spike** (the official RISC-V ISA simulator) or **QEMU** paired with **Perf**.

WRITE THE BASELINE (SCALAR) AI OPERATOR

This is a software implementation, the baseline "unoptimized" car driving on a standard highway.

- Implement a simple Matrix Multiplication or Softmax algorithm in pure C.
- Compile it down to standard RISC-V assembly.

WRITE THE OPTIMIZED (VECTOR) AI OPERATOR

Rewrite the same operator utilizing **RISC-V Vector Intrinsic functions** (or raw assembly inline). This forces the compiler to map the math onto vector registers (v0-v31), allowing single instructions to process multiple data streams simultaneously (SIMD).

PROFILE THE MICROARCHITECTURE

- Run both binaries through the simulator with tracing flags enabled.
- Extract raw hardware execution metrics

  - **Instruction Count:** Did vectorization dramatically lower the number of instructions fetched?
  - **IPC (Instructions Per Cycle):** Are the vector pipelines stalling waiting for memory?
  - **Cache Misses:** How cleanly did the data layout fit into the L1 cache?

THANK YOU